المطعم: السؤال والتحليل

هذا الملف للتفكير فقط: **نقرأ المسألة، ونرسمها، ونقرر ما نحتاجه ولماذا**. الكود في ملف آخر. قبل أن تكتب أي سطر، افهم لماذا كل شيء هنا موجود.

1 نص السؤال

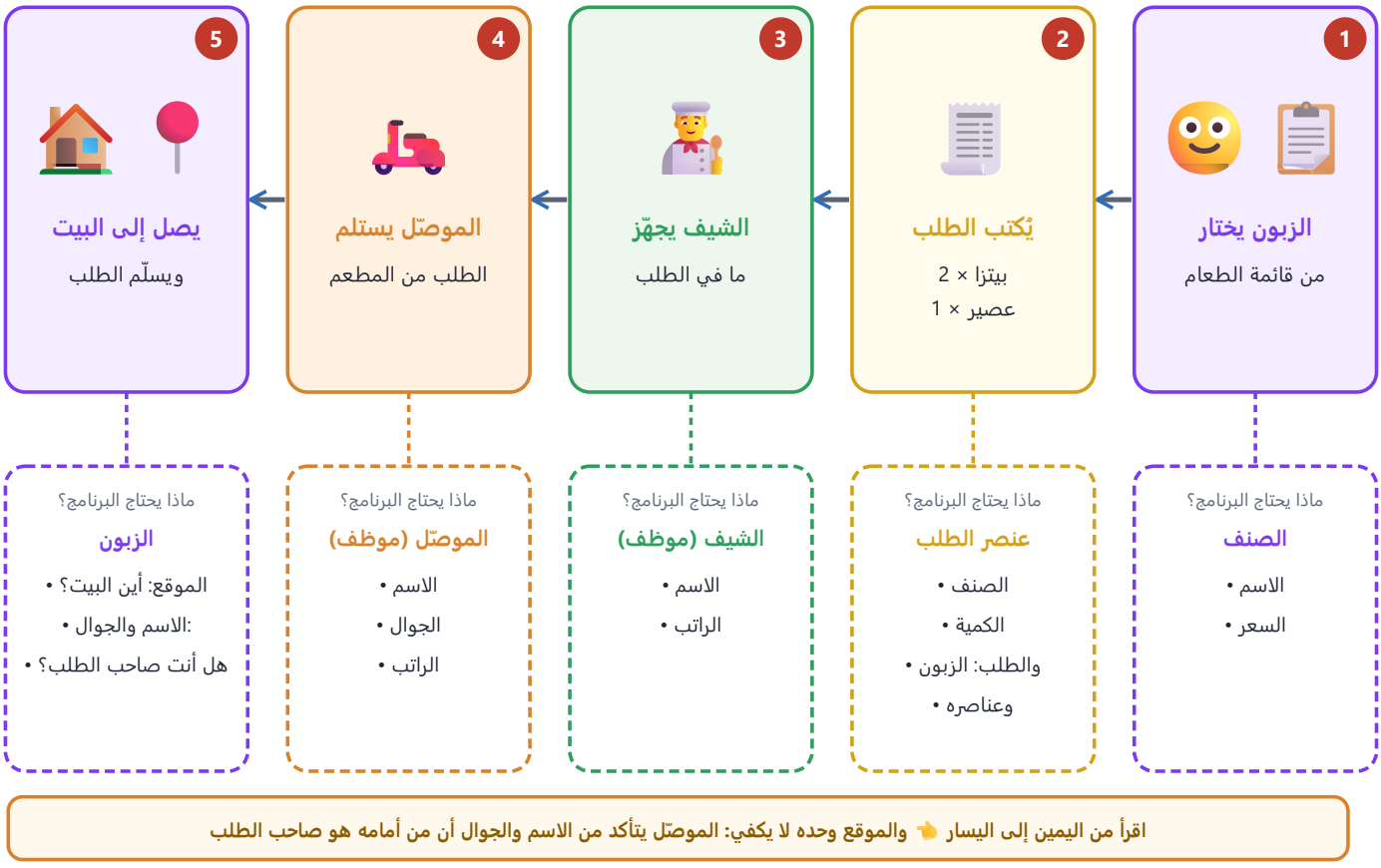
عندي **مطعم** وأريد برنامجاً يديره. في المطعم **أصناف** من ثلاثة أنواع: **وجبات** و**مشروبات** و**حلويات**، ولكل صنف اسم وسعر.

في المطعم **أشخاص** من نوعين: **زبائن** و**موظفون**، والموظفون إما **شيف** يجهّز الطعام أو **موصّل** يوصّل الطلبات.

الزبون **يدفع** ثمن طلبه، والموظف **يُدفع له** راتب. والزبون يملأ **طلباً** من عناصر (صنف وكمية)، و**الموصّل** يوصّل الطلب إلى الزبون.

2 ارسم المسألة كأنها مطعم حقيقي

قبل المخطط، تخيّل المطعم: من فيه؟ ماذا يفعلون؟ وماذا يمرّ بينهم؟ كل مربع علوي خطوة من طلب واحد (اقرأها من اليمين إلى اليسار)، والمربع المنقط تحته يقول **ما الذي يحتاجه البرنامج** في تلك الخطوة:



#	ماذا يحدث؟	من يعمل؟
1	الزبون يقرأ القائمة ويختار	الزبون ← الأصناف
2	يُكتب الطلب: كل صنف مع كميته	الطلب وعناصره
3	الشيف يجهّز ما في الطلب	الشيف
4	الموصّل يستلم الطلب	الموصّل
5	الموصّل يصل إلى بيت الزبون ويسلّمه	الموصّل ← الزبون

3 كيف نعرف أن الشيء يصبح صفاً؟

خذ كل شيء رأيته في الصورة واسأله **سؤالين فقط**:

1) ما هي معلوماته؟ (أشياء يحملها)

2) ماذا يفعل؟ (أعمال يقوم بها)

إن كان عنده **معلومات وأعمال** فهو **صف**. وإن كان هو نفسه **مجرد معلومة** عن شيء آخر فهو **خاصية**.

الشيء	1) ما معلوماته؟	2) ماذا يفعل؟	القرار
الزبون	اسم، جوال، إيميل، موقع، نقاط	يدفع	🔵 **صف**
الشيف	اسم، جوال، إيميل، راتب	يجهّز الطعام	🔵 **صف**
الصنف	اسم، سعر	يعطي سعره بعد التخفيض	🔵 **صف**
عنصر الطلب	الصنف، الكمية	يحسب مجموعه	🔵 **صف**
الطلب	الزبون، عناصره	يحسب المجموع	🔵 **صف**
الاسم	لا شيء! هو نص فقط	لا شيء	🟢 **خاصية** (اسم الزبون مثلاً)
السعر	لا شيء! هو رقم فقط	لا شيء	🟢 **خاصية** (سعر الصنف)
الموقع	هنا: نص بسيط (اسم الحي)	لا شيء	🟢 **خاصية** (عنوان الزبون)

ملاحظتان:

- **الموقع** لو كان فيه (مدينة وشارع ورقم بناية) ويفعل شيئاً (يحسب المسافة) لصار صفاً. لكنه هنا مجرد نص، فنبقيه خاصية ليبقى التمرين بسيطاً.
- **المطعم** نفسه لا نصنع له صفاً، لأن القائمة والطلبات تكفي لقصتنا.

4 الواجهة: الفكرة الأهم في التمرين

أولاً: المشكلة التي تحلها الواجهة

في قصتنا: **المشروب والحلوى** يمكن تخفيض سعرهما في العروض، أما **الوجبة** فلا. السؤال: أين نكتب "التخفيض"؟ لنجرّب ثلاثة حلول:

الحل	ماذا يحدث؟	الحكم
1) نكتب تابع التخفيض في الأب `MenuItem`	سيرثه الجميع بما فيهم **الوجبة**. فنضطر أن نكتب لها تابعاً بلا معنى (تخفيض وجبة؟)	❌ خطأ
2) نضع خاصية في الأب: "يقبل تخفيضاً؟ نعم / لا"	تعمل، لكن لا شيء يضمن أن من قال "نعم" كتب طريقة حساب السعر. ونسأل `if` في كل مكان	⚠️ ضعيف
3) نصنع **واجهة** `Discountable`	من يقبل التخفيض يلتزم بكتابة التابع، ومن لا يقبل لا يظهر عنده شيء. والمترجم يتأكد من الالتزام	✅ الصحيح

ثانياً: ما هي الواجهة بالضبط؟

في الواجهة	مثال
اسم يصف قدرة (غالباً ينتهي بـ `able` أو `er`)	`Discountable` = "عليه تخفيض"
توابع بلا جسم: اسم التابع فقط، وهو وعد	`priceAfterDiscount()` بلا كود
لا خصائص ولا كود جاهز	لا اسم ولا سعر هنا

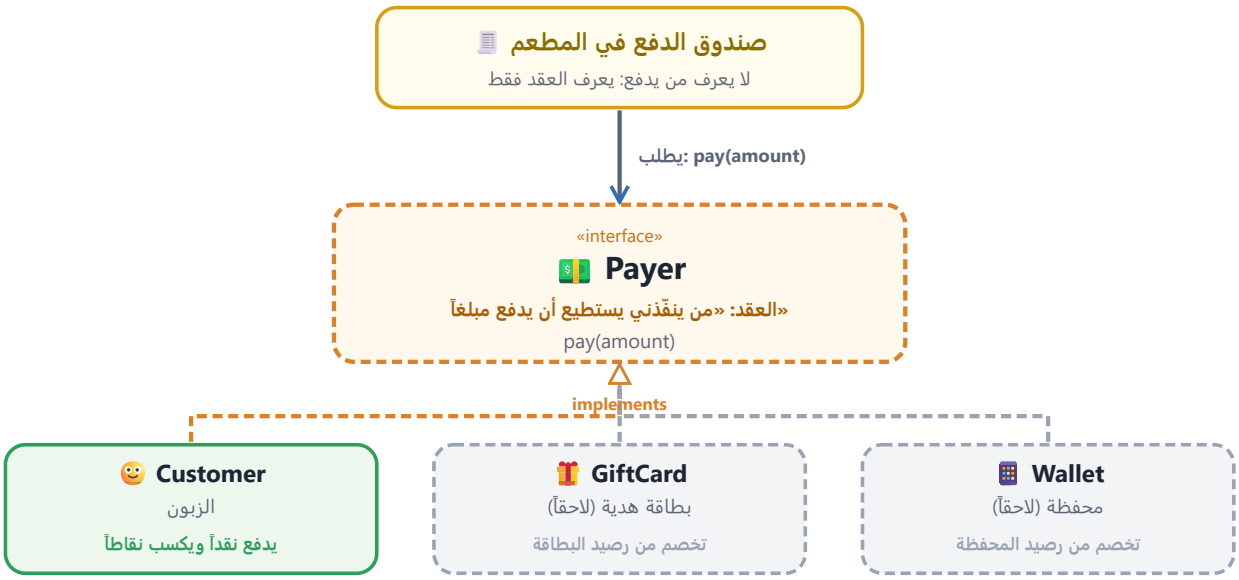
والصف الذي يقول `implements Discountable` **يلتزم** بكتابة التابع بطريقته. هكذا يعرف الجميع أن أي صنف عليه "ختم التخفيض" عنده هذا التابع.

ثالثاً: قواعد صغيرة تريحك

- لا يمكن عمل `new Discountable()`: الواجهة وعد وليست غرضاً.
- الصف يرث **أباً واحداً** فقط، لكنه ينفّذ **واجهات كثيرة** (مثلاً مشروب نباتي يقبل التخفيض: واجهتان).
- نقول عن الصف: **"هو"** عند الوراثة، و**"يستطيع"** عند الواجهة.
- الابن يرث واجهات أبيه: كتبنا `Payable` مرة في `Employee` فصار الشيف والموصّل يملكانها.

رابعاً: مثال الدفع بالتفصيل

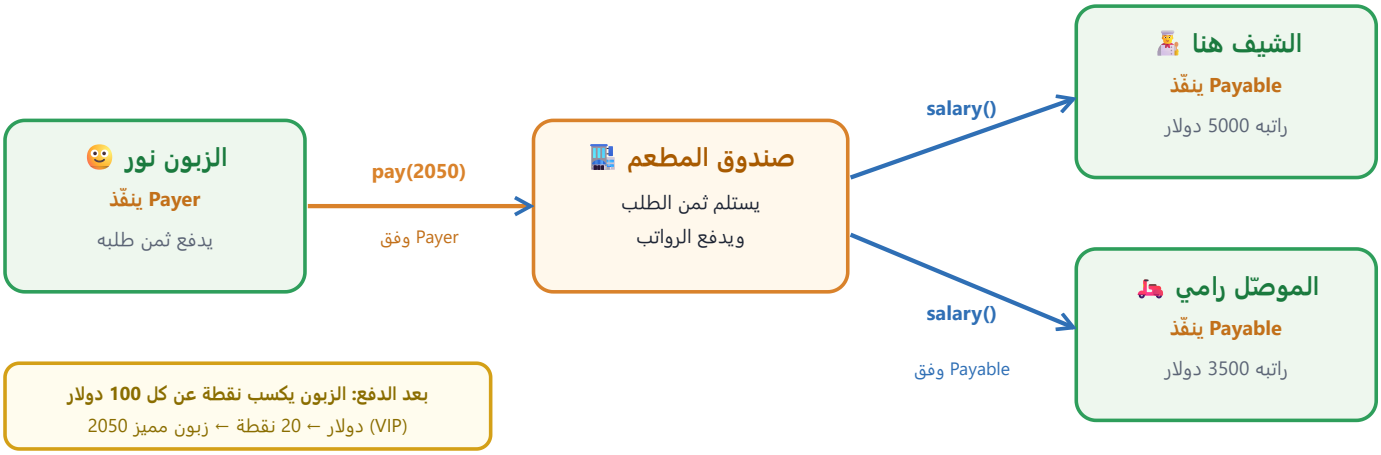
الواجهة مثل مقبس الكهرباء. المقبس لا يهمه أي جهاز: مروحة أو شاحن أو غلاية. يهمه فقط أن يكون للجهاز **قابس بالشكل المناسب**. كذلك واجهة `Payer` لا تهمها من يدفع: يهمها فقط أن يكون عنده تابع `pay`.



ماذا نتعلم من الصورة؟

- الصندوق يتعامل مع **العقد** (`Payer`) وليس مع الزبون.
- كل صف ينفّذ العقد **يكتب** `pay` **بطريقته**: الزبون يكسب نقاطاً، والبطاقة تخصم من رصيدها. العقد يقول **ماذا**، والصف يقول **كيف**.
- المربعان **الرماديان** ذوا الإطار المتقطع (بطاقة الهدية والمحفظة) صفوف **غير موجودة الآن**، وضعتها كأمثلة لما قد نضيفه مستقبلاً. الزبون وحده هو الموجود حالياً.
- لو أضفنا واحداً منها غداً فلن نغيّر الصندوق أبداً، لأنه يتعامل مع العقد فقط.

الآن قصة المال في مطعمنا كلها: الزبون **يدفع** ثمن طلبه، والمطعم **يدفع** رواتب موظفيه. هذان اتجاهان مختلفان، فلكل منهما عقد:



العقد	معناه	من ينفّذه	من لا ينفّذه ولماذا	مثال من البرنامج
`Payer` `pay(amount)`	يدفع مالاً	الزبون	الموظف: لا يدفع ثمن الطلب	`Nour pays 2050$`
`Payable` `salary()`	يُدفع له راتب	الشيف والموصّل (عن طريق `Employee`)	الزبون: لا راتب له	`Hana: 5000$` ثم `Total salaries: 8500$`

كيف تعمل الواجهة؟ أربع خطوات

#	الخطوة	في مطعمنا
1	**نكتب العقد**: اسم وتابع بلا جسم (وعد فقط)	`Payer` فيه `pay(amount)`
2	**الصف يلتزم** بالعقد ويكتب التابع بطريقته	`Customer implements Payer` ويكتب `pay`
3	**من يحتاج الخدمة** يتعامل مع العقد فقط	الصندوق يقول: "أعطني أي `Payer` "
4	**نبدّل المنفّذ** دون أن نغيّر المستعمل	غداً نضيف `GiftCard` ولا نلمس الصندوق

تذكّر: الواجهة **لا فيها كود**، بل وعد: "سأكتب هذا التابع". الصف هو الذي يكتب الكود. ولهذا يستطيع صفّان مختلفان تماماً (زبون وبطاقة) أن يلتزما بالوعد نفسه ويعملا بطريقتين مختلفتين، وهذا هو **تعدد الأشكال** مرة أخرى.

5 ماذا نحتاج في كل صف؟ وليش؟

لا نكتب في الصف كل معلومة تخطر ببالنا. نكتب فقط **المعلومات التي سيستعملها البرنامج فعلاً**. فقبل أن تضيف أي خاصية اسأل نفسك: **أين سأستعمل هذه المعلومة؟** مثلاً: الموقع يستعمله الموصّل ليعرف أين يذهب، والكمية تستعملها الفاتورة لتحسب المجموع. وإن لم تجد من يستعملها فلا تكتبها.

الصف	الخاصية	ليش نحتاجها؟
شخص (للجميع)	الاسم	لنناديه ونعرف من هو
	رقم الجوال	للتواصل معه (الموصّل يتصل بالزبون عند الباب)
	الإيميل	لإرسال تأكيد الطلب والفاتورة
زبون	الموقع (العنوان)	**ليعرف الموصّل أين يذهب.** بدونه لا يمكن التوصيل
	نقاط الولاء	مكافأة الزبون الدائم، ونعرف منها الزبون المميز
موظف	الراتب	ليعرف المطعم كم يدفع كل شهر (للشيف والموصّل فقط)
صنف	الاسم	ليظهر في القائمة وعلى ورقة الطلب
	السعر	لحساب مجموع الطلب
وجبة / مشروب / حلوى	الوزن / الحجم / السعرات	معلومة تخص نوعاً واحداً فقط: لا مكان لها في الأب
عنصر الطلب	الصنف	ليعرف أي شيء طُلب
	الكمية	لأن الزبون قد يطلب القطعة مرتين أو ثلاثاً
طلب	الزبون	ليعرف الموصّل لمن هذا الطلب
	عناصره	ليعرف ماذا طُلب ويحسب المجموع

قاعدة: المعلومة المشتركة بين كل الأنواع تذهب إلى **الأب** (الاسم والجوال والإيميل للأشخاص، والاسم والسعر للأصناف). والمعلومة التي تخص نوعاً واحداً تبقى عند **الابن** (الوزن للوجبة، الحجم للمشروب، الموقع للزبون، الراتب للموظف).

6 الإنكبسوليشن (التغليف) Encapsulation: كيف نحمي المعلومات؟

الفكرة (Encapsulation): لا نترك أي أحد يكتب أي قيمة في خصائص الصف كما يشاء. نغلق الخصائص (`private`)، ونضع **حارساً** عند الباب (تابع `set`) يتحقق من القيمة قبل أن يقبلها.

مثل خزنة المطعم: لا أحد يمد يده إليها. الجميع يعطي المال لأمين الصندوق، وهو يتأكد أن المبلغ صحيح ثم يضعه.



أين طبّقنا الفكرة في مطعمنا؟

الصف	الخاصية	القاعدة (ما يحميه الحارس)	ماذا يحدث لو خالفها أحد؟
`MenuItem`	السعر	لا يكون سالباً	يصبح 0
`Employee`	الراتب	لا يكون سالباً	يصبح 0
`Person`	الإيميل	فيه `@` ليكون إيميلاً	يصبح `no email`
`Customer`	النقاط	**لا أحد يغيّرها من الخارج**: تزيد فقط حين يدفع الزبون	لا يوجد تابع `set` لها أصلاً
`OrderItem`	الكمية	أكبر من صفر (نضيفها في الواجب)	يُرفض العنصر

ثلاث خطوات نكررها لكل خاصية

#	الخطوة	لماذا؟
1	نجعل الخاصية `private`	ليغلق الباب: لا أحد من خارج الصف يكتب فيها مباشرة
2	نكتب تابع `get` إن أردنا أن يقرأها الآخرون	يقرأ الجميع القيمة لكن لا يغيّرها
3	نكتب تابع `set` **يتحقق** قبل أن يغيّر	هذا هو الحارس: يقبل الصحيح ويرفض الغلط

لا نكتب `set` لكل خاصية. اسأل: هل يجب أن يتغير هذا الشيء من الخارج؟ نقاط الزبون مثلاً لا يجب أن يغيّرها أحد بيده، فلا نكتب لها `set` . فالتغليف يعني أيضاً **أن نقرر ما يُترك للآخرين وما لا يُترك**.

لماذا لا نترك الخاصية `public` ونكتب السعر مباشرة؟

لأننا لا نستطيع حينها أن نمنع الغلط. لو كان السعر `public` لكتب أحدهم `price = -50` فصار عندنا صنف بسعر سالب، ومجموع الطلب يصبح خاطئاً، وربما يربح الزبون مالاً! ولا نعرف أي سطر في البرنامج كتب القيمة الغلط.

أما مع الحارس فكل التغييرات تمر من **مكان واحد**: إن أردنا تعديل القاعدة (مثلاً السعر لا يتجاوز 10000) نغيّره في تابع واحد وينتهي الأمر.

7 المخطط: صورة التحليل كله

أولاً: خمس كلمات سنراها في المخطط، بأمثلة من حياتك

قبل أن تنظر إلى الأسهم، افهم معنى كل كلمة بمثال عادي. بعدها سترى نفس الأمثلة داخل المطعم.

وراثة — "هو"

ببساطة	الابن يأخذ من أبيه كل ما يشترك فيه الجميع، ويضيف شيئاً يخصه وحده.
مثال من حياتك	**السيارة والدراجة النارية**، كلاهما **مركبة**. لكل مركبة لون وسرعة (نكتبهما مرة واحدة في "مركبة")، وللسيارة أبواب أما الدراجة فلها مقود فقط.
السؤال	هل A **هو** نوع من B؟
في مطعمنا	الشيف **هو** موظف.

واجهة — "يستطيع"

ببساطة	ورقة شروط (عقد): كل صف ينفّذها يلتزم أن يفعل شيئاً معيناً، حتى لو لم تكن بين الصفوف قرابة.
مثال من حياتك	**بطاقة دخول النادي**: يحملها طفل وعجوز ومدرب. ليسوا أقارب، لكن كلهم **يستطيعون** الدخول.
السؤال	هل A **يستطيع** أن يفعل شيئاً يشاركه فيه غيره؟
في مطعمنا	المشروب والحلوى **يستطيعان** أن يُخفَّض سعرهما.

تركيب — "ملك لي ويموت معي"

ببساطة	الجزء يُصنع **داخل** الكل، ولا معنى له بدونه، ويزول إن زال الكل.
مثال من حياتك	**فاتورة السوبرماركت** وبنودها: البنود تُكتب مع الفاتورة. لو مزّقت الفاتورة فلا قيمة لأي بند.
السؤال	هل الكل **بنى** الجزء بنفسه، ويموت الجزء معه؟
في مطعمنا	**الطلب** وعناصره ("بيتزا × 2").

تجميع — "أحمله وهو يعيش بدوني"

ببساطة	الكل يحمل أشياء كانت **موجودة قبله** وتبقى بعده. وقد يشاركه فيها غيره.
مثال من حياتك	**فريق كرة القدم** واللاعبون: اللاعب كان موجوداً قبل الفريق. لو حُلّ الفريق يبقى اللاعب ويلعب في فريق آخر.
السؤال	هل الجزء **جاء من الخارج** ويبقى بعدي؟
في مطعمنا	**الطلب** والزبون: الزبون موجود قبل الطلب وله طلبات كثيرة.

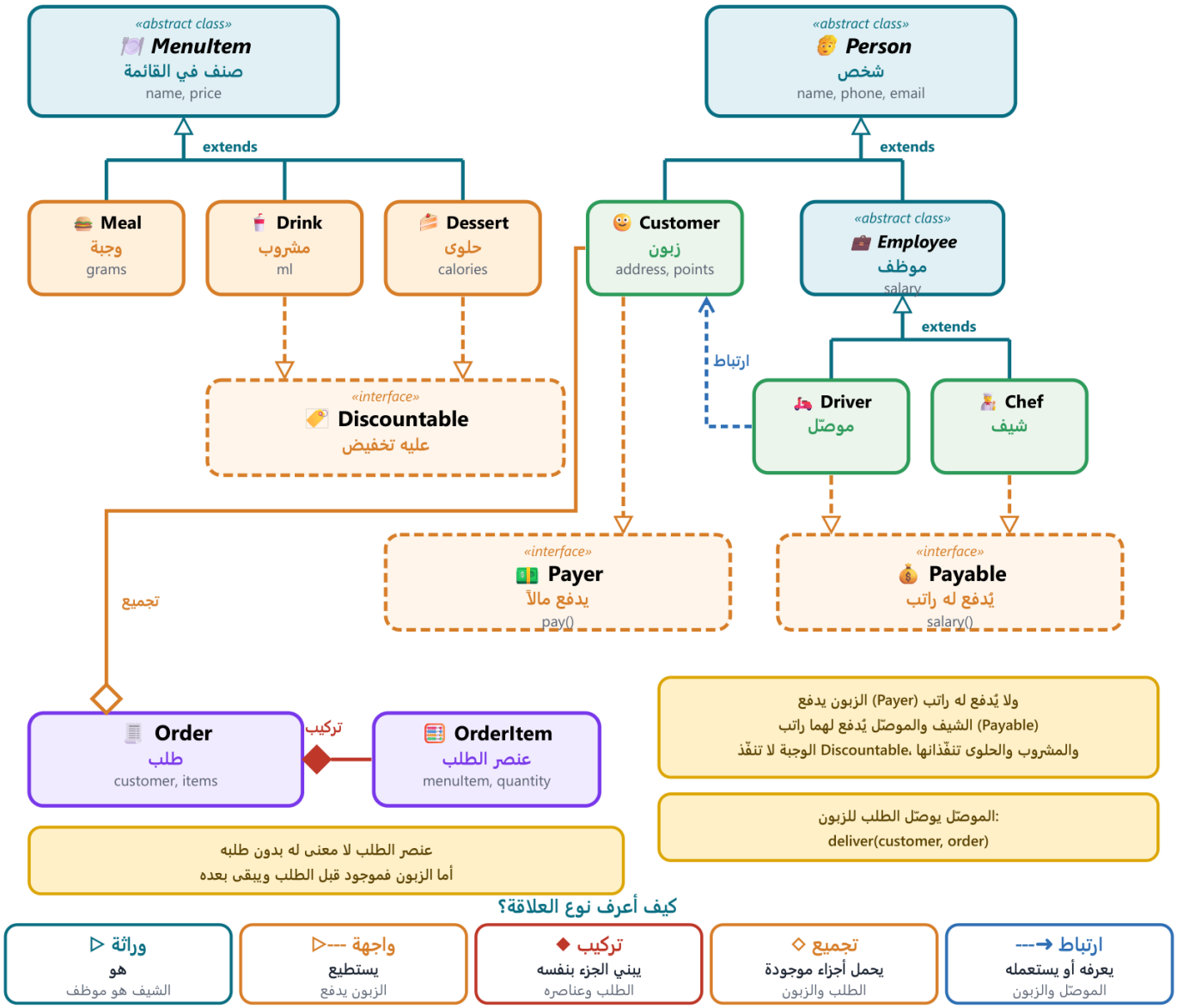
ارتباط — "أعرفه أو أستعمله فقط"

ببساطة	شيئان يلتقيان **لحظة** ثم يفترقان. لا يملك أحدهما الآخر ولا هو جزء منه.
مثال من حياتك	**ساعي البريد** والمرسَل إليه: يسلّمه الرسالة ويمضي. لا يحتفظ به ولا يملكه.
السؤال	هل **يستعمله لحظياً** دون أن يملكه؟
في مطعمنا	**الموصّل** والزبون: يوصّل له الطلب وينتهي الأمر.

تذكّر بثلاث كلمات: الوراثة **"هو"**، والواجهة **"يستطيع"**. وإن كان الشيء **جزءاً** فاسأل: **من صنعه؟** الكل بنفسه (تركيب)، أم جاء من الخارج (تجميع)؟ وإن لم يكن جزءاً وإنما **يلتقيه لحظة** فهو ارتباط.

ثانياً: المخطط

انظر الآن إلى المخطط. كل صندوق صف، وكل سهم علاقة من الخمس. ابحث عن كل شكل وتذكّر مثاله.



لاحظ: كل ما ناقشناه يظهر في المخطط. الأب في الأعلى، والأبناء تحته، والواجهات (الصناديق المنقطة) لما يستطيع بعضهم أن يفعله، والأسهم تبيّن من يعرف من.

كيف أقرأ الأسهم؟

الشكل	اسمه	معناه بكلمة
خط متصل ينتهي بمثلث فارغ ▷	**وراثة**	"هو": الابن هو نوع من الأب (السهم يشير إلى الأب)
خط منقط ينتهي بمثلث فارغ ---▷	**واجهة**	"يستطيع": الصف يلتزم بعقد
معين **ممتلئ** ◆ عند الكل	**تركيب**	"ملك لي ويموت معي"
معين **فارغ** ◇ عند الكل	**تجميع**	"أحمله وهو يعيش بدوني"
خط منقط بسهم عادي ---→	**ارتباط**	"أعرفه أو أستعمله فقط"

علاقات المخطط، واحدة واحدة

وراثة `Meal , Drink , Dessert → MenuItem`

القصة	الوجبة صنف في القائمة، والمشروب صنف، والحلوى صنف.
لماذا؟	اختبار "هو" ينجح للثلاثة. يشتركون في الاسم والسعر فنضعهما عند الأب مرة واحدة. ولكل واحد معلومة تخصه (الوزن، الحجم، السعرات) فتبقى عنده.
لو غيّرنا؟	لو جعلناها واجهة ("يستطيع") لما استطعنا مشاركة الاسم والسعر، لأن الواجهة لا تحمل خصائص.

وراثة `Customer , Employee → Person , Chef , Driver → Employee`

القصة	الزبون شخص، والموظف شخص، والشيف موظف، والموصّل موظف.
لماذا؟	الاسم والجوال والإيميل للجميع، فنضعها في `Person` . أما الراتب فللموظفين فقط، فنضعه في `Employee` ولا نضعه عند الزبون.
لو غيّرنا؟	لو جعلنا الشيف والموصّل يرثان من `Person` مباشرة لكرّرنا الراتب في كل منهما.

واجهة `Drink , Dessert ---▷ Discountable`

القصة	المشروب والحلوى يمكن تخفيض سعرهما في العروض.
لماذا؟	المشروب والحلوى يشتركان في قدرة ليست عند الوجبة. لو وضعنا "التخفيض" في الأب (`MenuItem`) لوجب على الوجبة أن تملكه أيضاً بلا معنى.
لو غيّرنا؟	لو جعلناه وراثة (حلوى هو مشروب؟!) لما صح اختبار "هو".

واجهة `Customer ---▷ Payer`

القصة	الزبون يدفع ثمن طلبه.
لماذا؟	الدفع قدرة تخص الزبون. والموظف لا ينفّذها، لأنه لا يدفع ثمن الطلب، بل يُدفع له.

واجهة `Chef , Driver ---▷ Payable`

القصة	الشيف والموصّل يُدفع لهما راتب.
لماذا؟	الزبون لا يُدفع له راتب، فلا ينفّذها. وفي الكود نكتبها مرة واحدة في `Employee` فيرثها الشيف والموصّل معاً (الابن يرث واجهات أبيه).
لو غيّرنا؟	لو كتبناها في `Person` لصار الزبون يُدفع له راتب أيضاً!

تركيب `Order ◆ OrderItem`

القصة	الطلب فيه عناصر: "بيتزا × 2"، "عصير × 1".
لماذا؟	الطلب هو الذي **يبني** العنصر بنفسه (`new` داخله)، والعنصر لا معنى له بدون طلبه، وكل عنصر في طلب واحد فقط. لو انتهى الطلب انتهت عناصره.
لو غيّرنا؟	لو بنينا العناصر خارج الطلب ثم مررناها له لصارت تجميعاً.

تجميع `Customer` ◇ `Order`

القصة	الطلب يخص زبوناً معيناً.
لماذا؟	الزبون **موجود قبل الطلب** (أُنشئ في مكان آخر) ويبقى بعده، وله طلبات كثيرة. الطلب يحمل "إشارة" إليه فقط.
لو غيّرنا؟	لو كان تركيباً لمات الزبون بمجرد انتهاء طلبه!

ارتباط `Driver` ⟶ `Customer`

القصة	الموصّل يوصّل الطلب إلى الزبون عند بيته.
لماذا؟	الموصّل لا يملك الزبون ولا هو جزء منه. يلتقيه لحظة التوصيل (`deliver(customer, order)`) ثم ينتهي الأمر. الزبون والطلب **وسيطان** للتابع، لا خاصيتان في `Driver` .
لو غيّرنا؟	لو خزّنه خاصية لصار الموصّل "مربوطاً" بزبون واحد دائماً.

ارتباط (غير مرسوم) `MenuItem` **يشير إلى** `OrderItem`

القصة	كل عنصر طلب يقول أي صنف طُلب.
لماذا؟	الصنف موجود في القائمة قبل الطلب ويبقى بعده، وعناصر كثيرة تشير إليه. فليس جزءاً من العنصر ولا يموت معه، ولهذا لم نرسمه سهم تركيب.

طريقة سريعة للتمييز بين الثلاثة المتشابهة: اسأل **من أنشأ الجزء؟** إن أنشأه الكل بـ `new` داخله فهو **تركيب**. وإن وصل من الخارج وبقي بعده فهو **تجميع**. وإن مرّ كوسيط لحظي فقط فهو **ارتباط**.

8 نقاش: أسئلة يطرحها الطلاب، وكيف نفكر فيها

لماذا التوصيل يأخذ الزبون والطلب معاً؟ ألا يكفي الموقع؟ 💡

التفكير الأول: "الموصّل يحتاج الموقع فقط، فنعطيه العنوان".

المشكلة: الموقع وحده لا يكفي. تخيّل أن يصل الموصّل إلى البناية ويقول شخص: **"هذا طلبي!"** وهو ليس صاحبه. لو لم يكن عند الموصّل اسم الزبون وجواله لا يستطيع أن يتأكد ممن يستلم. هذا احتيال حقيقي يحدث.

إذن نحتاج: (1) **الزبون**: ليعرف المكان ويتأكد من هويته ويتصل به. (2) **الطلب**: ليعرف ماذا يسلّم وكم المبلغ.

ولماذا يأتيان وسيطين للتابع، ولا نجعلهما خاصيتين داخل الموصّل؟

	خاصيتان داخل الموصّل	وسيطان للتابع (ما نختاره)
ماذا يعني؟	الموصّل "يحمل" زبوناً وطلباً دائماً، فكأنه يملكهما	الموصّل يستلم الزبون والطلب **لحظة التوصيل فقط**
مثال	رامي مربوط بنور دائماً، فكيف يوصّل لغيرها؟	رامي يوصّل اليوم لنور، وغداً لزبون آخر
بعد انتهاء التوصيل	يبقى الرابط	ينتهي الرابط
الحكم	❌ خطأ: الموصّل لا يملك الزبون	✅ صحيح: هذا هو **الارتباط**

الخلاصة: الارتباط لا يعني أن أحدهما يملك الآخر. يعني أنهما **يلتقيان لحظة ثم يفترقان**.

ملاحظة: الطلب نفسه يحمل الزبون، فتقنياً الطلب يكفي ليصل إلى الزبون. لكننا نمرّر الاثنين ليظهر الارتباط بوضوح، وليتحقق الموصّل أن "الزبون الذي أمامه" هو "صاحب هذا الطلب".

لماذا الطلب يحمل عناصر طلب (OrderItem)، ولا نضع مصفوفة أصناف داخله؟ 💡

التفكير الأول: "الطلب = قائمة أصناف، فنضع مصفوفة أصناف داخله وانتهينا".

وهناك طلاب يفكرون بطريقة ثانية: "كل صنف يطلبه الزبون طلب مستقل، وأضع **مصفوفة طلبات**". لنختبر الفكرتين معاً:

لنختبر الفكرة: الزبون يريد **بيتزا مرتين** و**عصيراً مرة**:

الفكرة	ما يحدث	المشكلة
مصفوفة أصناف فقط	[بيتزا، بيتزا، عصير]	نكرر الصنف بدل أن نقول "2". مع 20 قطعة يصبح الأمر سخيفاً، ولا نعرف الكمية إلا بالعدّ
مصفوفتان: أصناف وكميات	[بيتزا، عصير] و[2، 1]	الاثنتان مرتبطتان بالمكان فقط. لو حذفنا صنفاً ونسينا الكمية اختلطت الأمور وضاع الترتيب
مصفوفة طلبات: كل صنف طلب مستقل	طلب (بيتزا)، طلب (بيتزا)، طلب (عصير)	الزبون "طلب مرة واحدة" لكن عندنا ثلاثة طلبات! الاسم والعنوان يتكرران في كل طلب، ولا نعرف أيها من نفس الزيارة. والموصّل يوصّل ثلاث مرات بدل مرة واحدة، والزبون يدفع ثلاث مرات بدل مجموع واحد
عنصر طلب (صنف + كمية معاً)	[(بيتزا،2)، (عصير،1)]	لا مشكلة: كل صنف معه كميته في غرض واحد، والمجموع سهل: سعر × كمية

الخلاصة: "الطلب" في الحقيقة هو **زيارة واحدة** فيها أصناف كثيرة. فنحتاج صفاً لـ**الطلب كله** (الزبون ومجموعه)، وصفاً لـ**كل صنف فيه** مع كميته. وهكذا نحصل على الطلب الواحد والعناصر الكثيرة. والعنصر (الصنف مع كميته) له معلومات خاصة به، فيستحق أن يكون **صفاً**. وهو لا يعيش بدون طلبه، فهو **تركيب**.

لماذا الكمية ليست داخل `MenuItem` ؟ 💡

اسأل نفسك أولاً: **من الذي يقرر كل معلومة؟** فهنا شخصان مختلفان، كل واحد يقرر شيئاً:

	🏪 صاحب المطعم	🙂 الزبون
ماذا يقرر؟	ماذا **يبيع** المطعم	ماذا **يشتري** من هذا الذي يُباع
القرارات	أي أصناف في القائمة، واسم كل صنف، وسعره	أي صنف يختار، وكم قطعة منه
متى يقرره؟	مرة واحدة **قبل** فتح المطعم	في كل مرة يأتي، ويختلف من زبون لآخر
أين نكتبه في البرنامج؟	`MenuItem` : الاسم والسعر	`OrderItem` داخل `Order` : الصنف والكمية

مثال: ثلاثة زبائن جاؤوا اليوم والبيتزا في قائمة المطعم:

الزبون	ماذا اختار؟
سارة	بيتزا × 2
أحمد	بيتزا × 1
ريم	لم تطلب بيتزا

صاحب المطعم لم يغيّر شيئاً في القائمة: البيتزا نفسها بسعرها نفسه. لكن الزبائن الثلاثة اشتروا كميات مختلفة. **الكمية قرار الزبون، وليست صفة من صفات البيتزا.**

ولو وضعنا الكمية داخل `MenuItem` ؟ فالبيتزا ستحمل رقماً واحداً فقط، فأي كمية نكتب؟ 2 (سارة) أم 1 (أحمد) أم 0 (ريم)؟ كل زبون يغيّر الرقم فيتغير عند الجميع! لهذا الكمية تذهب مع **الطلب**، حيث لكل زبون عناصره.

> **قاعدة:** ما يقرره **صاحب المطعم** يبقى في القائمة (`MenuItem`). وما يقرره **الزبون** يذهب إلى الطلب (`Order` و `OrderItem`).

لماذا الطلب يحمل الزبون (تجميع) وليس الزبون يحمل طلباته؟

اتجاه العلاقة يتبع **الحاجة**: الطلب يحتاج زبوناً ليُوصَّل له. أما الزبون فلا يحتاج طلباته في هذا التمرين. والزبون موجود قبل الطلب ويبقى بعده، فهو **تجميع** وليس تركيباً.

لماذا الشيف والموصّل صفان، وليس موظفاً واحداً فيه خاصية "نوع الوظيفة"؟

لأن لكل منهما **تصرفاً مختلفاً**: الموصّل يوصّل، والشيف يجهّز. لو جمعناهما في صف واحد لاحتجنا إلى `if` في كل مكان لنعرف من هو. بالوراثة يعرف كل غرض نفسه، ونكتب حلقة واحدة على الجميع.

لماذا نفصل **الزبون** و**الموظف** رغم أنهما شخصان؟

يشتركان في الاسم والجوال والإيميل (فنضعها في `Person` مرة واحدة). ويختلفان: الزبون له موقع ونقاط ويدفع، والموظف له راتب ويُدفع له. فنصنع لكل منهما صفه.

لماذا `Payer` واجهة؟ ألا نكتب تابع الدفع داخل `Customer` مباشرة؟ 💡

التفكير الأول: "الزبون هو الذي يدفع، فأكتب `pay` داخله وانتهينا. لماذا واجهة؟"

الجواب الصريح: فكرتك **تعمل فعلاً**! لو كان في المطعم زبون واحد فقط يدفع، لكان التابع داخل `Customer` كافياً. لكن اسأل نفسك: **ماذا لو صار غير الزبون يدفع؟**

مثلاً: **بطاقة هدية** تدفع عن الزبون، أو **محفظة إلكترونية**، أو **شركة** تدفع لموظفيها. ليس أي منها "زبوناً"، لكنها كلها **تستطيع الدفع**.

	التابع داخل الزبون فقط	واجهة `Payer`
من يدفع؟	الزبون فقط	أي صف يقول "أنا أستطيع الدفع"
تابع الحساب (الدفع عند الصندوق)	يقبل `Customer` فقط. وكل نوع جديد يحتاج تابعاً جديداً	يقبل **أي** `Payer` : زبون، بطاقة هدية، محفظة، دون أن نغيّره
قائمة المدفوعات	لا نستطيع جمع الزبون والبطاقة في قائمة واحدة	`ArrayList<Payer>` تجمعهما، وحلقة واحدة تنادي `pay`
الضمان	لا يوجد: قد ينسى أحد كتابة التابع	المترجم يرفض أي صف يقول "أنا `Payer`" ولا يكتب `pay`

الفكرة: الواجهة تفصل بين **"ماذا نريد"** (أن يدفع شيء ما) و**"من" و"كيف"** (زبون بالنقد، أو بطاقة، أو نقاط). برنامج الدفع يقول فقط: "أعطني أي شيء يدفع"، ولا يهمه من هو.

وفي تمريننا بالذات؟ صراحةً: `Payer` ينفّذها `Customer` وحده الآن. نكتبها **للتدرّب على الفكرة**، ولأننا نعرف أن المطعم سيكبر ويدخل عليه من يدفع غير الزبون. في المشاريع الحقيقية نبدأ بالواجهة حين يوجد صفان أو أكثر يشتركان في القدرة، أو حين نتوقع ذلك قريباً. وعلى المنوال نفسه: `Payable` ينفّذها الشيف والموصّل (اثنان فعلاً).

لماذا نحتاج واجهات؟ ألا تكفي الوراثة؟

الوراثة تقول "هو"، أما الواجهة فتقول "يستطيع". المشروب والحلوى **يقبلان التخفيض** بينما الوجبة لا، رغم أن الثلاثة أصناف. والزبون **يدفع** بينما الموظف **يُدفع له**، رغم أنهما أشخاص. هذه الفروق لا يحلها الأب، فنضع لها عقوداً تأخذها الصفوف التي تناسبها فقط.

9 نبدأ بالحل: من أين؟ وبأي ترتيب؟

بعد أن فهمنا المسألة وحلّلناها، لا نكتب كل الصفوف دفعة واحدة. نمشي **مراحل صغيرة**، وفي كل مرحلة **نكتب شيئاً ثم نجربه** قبل أن ننتقل. هكذا نكتشف الخطأ حين يكون صغيراً.

#	المرحلة	ماذا نكتب؟	ماذا نجرّب؟	ماذا يجب أن نرى؟
1	**الأصناف**	`MenuItem` (مجرد) ثم `Meal` و `Drink` و `Dessert` مع `toString()`	في `main`: نبني صنفاً من كل نوع ونطبع كلاً منها	ثلاثة أوصاف مختلفة
2	**التخفيض**	الواجهة `Discountable`، وينفّذها المشروب والحلوى	حلقة على الأصناف بـ `instanceof`	السعر بعد التخفيض للمشروب والحلوى، والوجبة تُتخطّى
3	**الأشخاص**	`Person` (مجرد) ثم `Customer` و `Employee` ثم `Chef` و `Driver` (مع حماية السعر والإيميل)	نبني شخصاً من كل نوع، ونجرب إيميلاً بلا `@`	أوصاف الأشخاص، والإيميل الغلط صار `no email`
4	**المال**	الواجهتان `Payer` و `Payable`	حلقة على الأشخاص لجمع الرواتب	راتب الشيف والموصّل فقط، والزبون لا يظهر
5	**الطلب**	`OrderItem` ثم `Order` (`addItem` و `total`)	نبني طلباً من صنفين ونطبع المجموع	مجموع صحيح (سعر × كمية)
6	**التوصيل**	`deliver(customer, order)` في `Driver`	نوصّل الطلب السابق لزبون	جملة فيها اسم الموصّل والزبون وعنوانه
7	**القراءة من المستخدم**	`Scanner` في `main`	نستبدل الأغراض التي بنيناها يدوياً بما يكتبه المستخدم	نفس النتائج لكن من الكيبورد

لماذا هذا الترتيب؟ نبدأ بما لا يعتمد على غيره (الأصناف والأشخاص)، وننتهي بما يعتمد على الجميع (الطلب والتوصيل). و `Scanner` آخراً لأن الكتابة اليدوية في كل تجربة تُتعب، فنجرّب بأغراض جاهزة ثم نربط المستخدم في النهاية.

وإن علقت في مرحلة؟

- ارجع إلى **المخطط**: ما الصف الذي يخص هذه المرحلة؟ وما علاقته بغيره؟
- اكتب في ورقة: **ماذا أريد من هذا التابع؟** (المدخلات والنتيجة) قبل أن تكتبه.
- إن ظهر خطأ ترجمة اقرأ **أول سطر** فيه: غالباً هو الأهم (اسم غير موجود، أو واجهة لم نكمل تابعها).
- شغّل بعد **كل صف** وليس بعد عشرة.

الخلاصة: قبل أي سطر كود اسأل: **هل هو؟** (وراثة) **هل يستطيع؟** (واجهة) **من يبنيه؟ من يحمله؟ من يستعمله فقط؟** (تركيب، تجميع، ارتباط). إن عرفت الجواب صار الكود سهلاً.